

V5.5

Product Definition · Canonical

AI-Native Thematic Research Loop Infrastructure

Theme → Thesis → Scenarios ↔(N:M)↔ Baskets → Monitor & Evolve

取代v5.4 Canonical · 基于V5.5 Update Proposal (已批准)

Internal · Canonical Reference

零、Orientation · 如何阅读本文档

本文档是什么

本文档是 V5.5 Framework 的 **canonical 产品定义**——是 framework 所有后续决策、产品设计、工程实现的权威参考。任何与本文档冲突的早期材料（v5.0-v5.4 系列）均已 supersede。

V5.5 相对 V5.4 的两个结构性修正

修正 1（Loop 顺序）· Scenarios 从 loop 第 5 段移到第 3 段，紧跟 Thesis。真实 PM 认知流程是 thesis 写完后先穷举 scenarios 再决定 basket，不是反向。AI 的最大发力点也应该在 scenarios expansion，放末端会错过 PM 决策的核心时机。

修正 2（Basket 独立 + N:M）· Basket 是 PM 唯一可量化的 performance 指标（PnL、Sharpe、attribution 都挂在 Basket 上），必须作为 first-class object 独立存在。Scenarios 与 Baskets 是 N:M 关系——一个 basket 可表达多个 scenarios，一个 scenario 可衍生多个 basket 表达。

本文档结构

- **Chapter 1 · Framework 概览** — 定位、目标用户、核心原则、数据模型
- **Chapter 2-6 · Stage 1-5 详细定义** — 每个阶段：定义 / 对象 / 状态机 / AI function / 产品设计
- **Chapter 7 · AI-Native 设计原则** — AI 在框架中的角色总纲，避免局部优化导致 drift
- **Chapter 8 · Forks 作为 v5.5 的 Retail 实现** — 产品决策 vs framework 决策的边界
- **Chapter 9 · Schema & API 规范** — 工程实现参考

一、Framework 概览

1.1 Framework 是什么

V5.5 是一个 **面向 thematic investment research 的 AI-native loop infrastructure**。它不是市场预测引擎，不是 quant signal generator，不是 portfolio optimizer。它的 job 是：

- 结构化 (structure) PM 的思考——让 thesis / scenarios / baskets 成为可查询、可回溯、可审计的 first-class objects
- 追踪 (track) 思考在时间维度的演化——每个 scenario 被验证或证伪、每个 basket 的 PnL 归因、每次 evolve 的 lineage 保留
- 重放 (replay) PM 的判断——回头看 "当时是怎么想的、触发了什么、为什么 evolve"，让历史经验可复用

V5.5 不预测市场。它是 PM + AI 协作形成、测试、表达、追踪、进化 thesis 的完整生命周期 infrastructure。

1.2 目标用户

Primary user	专业 PM (buy-side)	关心什么
机构 PM	对冲基金、共同基金、family office 的 sector PM	Performance attribution、风险归因、lineage audit
独立 fund manager	管理 \$10-100M 规模的独立 fund	Framework 作为单 PM 的 operating system
research team lead	对冲基金的 head of research / CIO	跨 PM 的 research standardization + knowledge retention

Retail 用户 (fintwit、aspiring buy-side、semi-pro) 不是 Framework 的 primary user。这些用户是 Forks (作为 v5.5 的 retail 实现) 的目标用户，产品形态不同。见 Chapter 8。

1.3 三条核心原则

原则 1 · Thesis is commodity, scenarios are alpha

共识论断任何专业 PM 都能得出。 "AI 利好算力" 是共识；"美联储转鸽" 是共识；"中国房地产出清" 是共识。这些 thesis 层的判断不产生 alpha——所有 PM 都能在同一时间做出同样的 thesis。

Alpha 在 scenarios 的分叉。 同一个 "AI 利好算力" thesis 下，谁能预见 custom silicon (TPU、Trainium) 会分走蛋糕？谁能识别 bear scenario (模型效率突破让算力需求下滑) 下的非对称机会？Scenarios 是区分 PM 观点质量的唯一维度。

V5.5 把 Scenarios 放在 loop 第 3 段 (紧跟 Thesis)，就是承认这个事实——*scenarios 是 framework 的 center of gravity*。AI 最大的价值也在这里 (expand option space)，所以 AI 必须在这个阶段最 active。

原则 2 · AI expands option space, PM makes choices

AI 永远是 candidates generator, 不是 decision maker。 AI 在每个阶段的 job 是扩展 PM 的视野、挖掘未考虑的因子、起草 PM 可能忽略的分支。PM 的 job 是选择、深入、决定。

这条原则的具体表现：

- **AI 不替 PM 润色 thesis 语法**——thesis 是 PM 的认知表达，AI 只 sanity check (因果链是否闭环、证伪条件是否具体)
- **AI 不替 PM 选 scenario**——AI 起草候选，PM adopt / dismiss
- **AI 不替 PM 定 basket weights**——AI 推荐 candidate instruments + 显示历史相似 baskets，PM 决定 weights 和 entry rules
- **AI 不自动 invalidate scenario**——除非 PM 明确定义的 falsification condition 被 trigger 命中，否则 scenario 状态变化需要 PM 确认

原则 3 · Basket is the only quantifiable performance unit

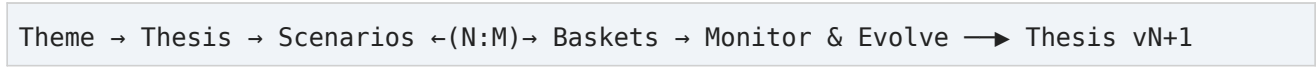
Theme、Thesis、Scenarios 都是 **qualitative 认知工件**——它们承载意图、判断、推理，但 **无法 backtest、无法 attribute PnL、无法 audit**。

唯一可以承载 quant 信号的对象是 Basket。所有 PM 关心的可量化指标——

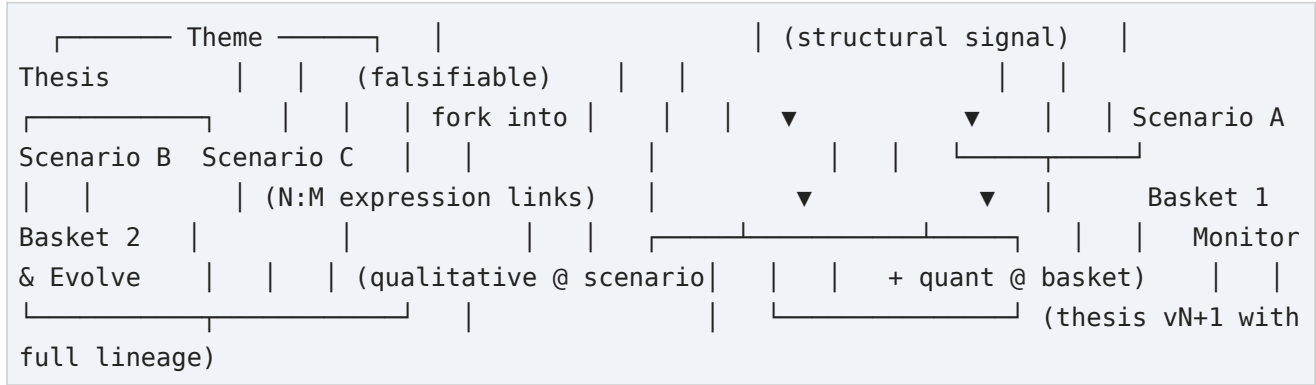
PnL、Sharpe、turnover、factor exposure、相关性、capacity、historical attribution——全部归 Basket。

因此 V5.5 Framework 必须把 Basket 作为 *first-class object* 独立定义 (不内嵌在 Scenarios 里)，并通过 N:M 关系链接到 Scenarios。这让 framework 既能承载 qualitative judgment (在 Scenarios 层)，又能对接 quantitative reality (在 Baskets 层)。

1.4 Loop 结构



流程示意



1.5 数据模型总览

对象	关键字段	性质	关系
Theme	name, state, trigger_context	Qualitative	1:N → Thesis
Thesis	claim, causal_chain, falsification, parent_theme_id, version, state	Qualitative	1:N → Scenarios, N:1 ← Theme
Scenario	narrative, trigger_signals, parent_thesis_id, pm_probability, state	Qualitative	1:N ← Thesis, N:M ↔ Baskets
Basket	instruments, weights, entry_rules, risk_budget, factor_exposure	Quantitative	N:M ↔ Scenarios (via expression_links)
ExpressionLink	scenario_id, basket_id, weight_contribution	Relation	Links Scenario ↔ Basket
TriggerEvent	scenario_id, signal_matched,	Event	N:1 → Scenario

	timestamp, severity		
PerformanceRecord	basket_id, timestamp, pnl, attribution_breakdown	Quantitative	N:1 → Basket

关键数据模型要点：(1) ExpressionLink 表是 N:M 关系的载体——多对多不是靠字段硬塞，是靠独立 linking table；(2) TriggerEvent 挂 scenario 层（qualitative），PerformanceRecord 挂 basket 层（quantitative），两种 monitor 数据分离；(3) Thesis 有 version 字段，evolve 时不覆盖，保留完整 lineage。

2.1 定义

Theme 是 framework 最上层的 topic container。它代表一个 PM 认为值得长期追踪的结构性议题（macro 趋势、sector shift、policy direction、consumer behavior change）。

Theme 不是 thesis。Theme 是"这个方向值得关注"，thesis 是"在这个方向下，我认为X 会发生（且可证伪）"。

2.2 Theme 对象

```
Theme {  id: UUID    name: string                                // "AI Infrastructure Buildout"
description: string      // 长文本，说明 theme 边界    state: enum
// emerging / active / peaking / decaying / dormant    trigger_context: JSON
// PM 标注的为何关注此 theme 的背景    related_themes: Theme[]      // 相关/对立的
themes    created_at, updated_at, author_id }
```

2.3 Theme 状态机

State	含义	进入条件	出口条件
emerging	刚被识别，证据有限	PM 创建 theme	积累 3+ 有效 theses → active
active	正在产生新 theses 和 scenarios	有 active theses 持续产出	3+ 月无新 thesis → peaking
peaking	共识已形成，alpha 衰减	PM 手动标注 or AI proactive suggestion	6+ 月无新 thesis → decaying
decaying	价值衰减期，仅追踪收尾	长期不活跃	12+ 月 → dormant
dormant	休眠，但保留 lineage	时间或 PM 标注	新 trigger 事件 → 重新激活

2.4 AI 在 Theme 阶段的 function



AI · Theme Radar AI 7×24 扫描 macro/sector signal (新闻、ETF flow、policy release、sector earnings) , proactively 建议: (1) 新 theme emergence (检测到未被 PM 追踪的结构性信号); (2) state transition (某 theme 6 个月无新 thesis, 建议标记 peaking); (3) cross-theme correlation (检测到两个 themes 在 trigger 层有 overlap, 建议 PM 审视)。



AI · Theme Sanity Check PM 创建新 theme 时, AI check: (1) 是否与已有 themes 重复 (语义相似度); (2) 边界是否清晰 (太宽 / 太窄); (3) 是否有 actionable 程度 (能不能 fork 出可证伪的 theses)。AI 给出 feedback, PM 决定是否采纳。

AI 不做什么

- 不自动创建 theme (PM 决定关注什么)
- 不自动 transition state (只能建议, PM 确认)
- 不润色 theme description (这是 PM 的表达)

2.5 产品设计要点

- **Theme dashboard**: 按 state 分类显示 (active / peaking / decaying / dormant) , quick filter, 每个 theme 显示 active theses count 和最近 trigger events
- **AI Suggestion Inbox**: Theme Radar 的 proactive suggestions 聚合, PM 每日或每周 review, 每个 suggestion 可 adopt / dismiss / later
- **Theme detail view**: 显示 theme 全 lineage (所有衍生 theses、scenarios、baskets) + historical trigger events timeline

3.1 定义

Thesis 是 theme 下 PM 做出的 **可证伪论断**。它必须满足三个条件：

- **有明确的 claim**——"A 公司股价会超过 B 公司" 不是 thesis；"AI 算力需求将从训练端转移到推理端" 是 thesis
- **有因果链**——为什么 claim 会成立？通过什么机制？
- **有证伪条件**——什么事件发生会证明 thesis 错？（e.g., "如果 NVDA Q4 data center 毛利率 > 75% 且 custom silicon 市场份额 < 15%，thesis 证伪"）

Thesis 是 commodity——同一个 theme 下，所有 sharp 的 PM 都能得出相似的 thesis。这不是贬义，是对 thesis 层的准确定位。Alpha 在下一阶段（Scenarios）。

3.2 Thesis 对象

```
Thesis {  id: UUID    parent_theme_id: UUID    claim: string                                // "AI
推理算力需求会转向 custom silicon"    causal_chain: string                                // 多段论证
falsification_conditions: [{    // 证伪条件    signal: string,                                // 可观测
信号    threshold: string,                // 阈值    }]    version:
int                                // 1, 2, 3... (evolve 时递增)    lineage: Thesis[]
// 前序版本 (v2 的 lineage 包含 v1)    state: enum                                // active /
evolving / invalidated / superseded    pm_confidence: float                        // 0-1 主观信心
(可选)    created_at, updated_at, author_id }
```

3.3 Thesis 状态机

State	含义	进入条件	出口条件
active	当前生效的 thesis 版本	PM 发布 thesis	新证据出现 → evolving
evolving	正在修订中，下一版本草案	AI 起草 vN+1 (based on trigger events), 或 PM 手动启动	PM 确认 → 生效为新 active
invalidated	falsification 被触发	Falsification condition 被 trigger 命中	可 revive → evolving → active (vN+1)
superseded	已被新版本取代	Evolve 完成, vN+1	只读（保留 lineage）

		active	
--	--	--------	--

Thesis lineage 是 V5.5 的重要特性——invalidated thesis 可以 revive 为 v2（保留 v1 lineage）。PM 不会因为一次证伪永远失去 thesis 的思考资产，而是可以带着历史经验重新 iterate。

3.4 AI 在 Thesis 阶段的 function

 **AI · Falsifiability Coach** PM 起草 thesis 时, AI check: (1) 因果链是否闭环 (论证是否 self-contained) ; (2) 证伪条件是否具体 ("NVDA 股价下跌" 不具体, "NVDA Q4 data center 毛利率 < 70%" 具体) ; (3) 是否与现有 theses 冲突 (在同一 theme 下)。AI 输出结构化 feedback, PM 决定是否修改。

 **AI · Cross-Thesis Conflict Detector** AI 检测新 thesis 是否与 PM 已有 active theses 隐含冲突。例: PM 已有 thesis "美联储转鸽", 又发布 "美元强势持续"——AI 提示两者 scenario 空间可能冲突, 建议 PM 审视。

 **AI · Thesis Evolution Drafter** 当 thesis 的某个 scenario 被 trigger, 且 PM 标记为 significant, AI 起草 thesis vN+1 草案: (1) 保留原有 claim 结构; (2) 整合新证据到 causal chain; (3) 更新 falsification conditions。PM review 后决定 accept / reject / modify。

AI 不做什么

- **不润色 thesis 语法**——thesis 是 PM 的认知表达, AI 语法改动会改变论断微妙之处
- 不自动发布 thesis (PM 必须手动 publish)
- 不自动 invalidate thesis (除非 falsification condition 明确命中 + PM 确认)
- 不判断 thesis "好坏" (AI 不做价值判断, 只做 structure check)

3.5 产品设计要点

- **Thesis Editor**: 三栏结构——claim / causal chain / falsification conditions。右侧实时显示 AI Falsifiability Coach feedback (可关闭)
- **Thesis timeline view**: 显示 thesis 从 v1 到 vN 的完整 lineage, 每次 evolve 标注 trigger event
- **Conflict alert**: 发布时 AI 检测冲突, PM 决定是否 proceed

Scenarios 是 V5.5 loop 的重心（center of gravity）。这是 framework 区分 alpha 的阶段，也是 AI 最活跃的阶段。AI 在这里的正确使用决定了整个 framework 的价值。

4.1 定义

Scenario 是 thesis 的一条 **可能未来路径**。同一个 thesis 可以 fork 出多个 scenarios（通常 3-6 个），每条 scenario 代表一种具体的 realization 方式。

一个 scenario 必须包含

- **narrative**——这条路径下会发生什么？为什么？通过哪些具体机制？
- **trigger signals**——可观测的 leading indicators。"如果发生 X，这条 scenario 被验证"
- **parent thesis**——明确关联到哪条 thesis（通过 parent_thesis_id）
- **PM subjective probability**——PM 对这条 scenario 的主观信心（0-1）。多条 scenarios 的概率之和不必等于 1（可以有 overlap 或 gap）

4.2 Scenario 对象

```
Scenario {  id: UUID  parent_thesis_id: UUID  narrative: string  // 长文本, 描述这条路径  trigger_signals: [{  // 可观测的 leading indicators  signal_type: enum,  // earnings / macro / industry / sentiment  description: string,  threshold: string,  current_status: enum,  // pending / triggered / disproved  last_checked: timestamp,  }]  pm_probability: float  // PM 主观概率  state: enum  // active / triggered / invalidated / obsolete  ai_generated: bool  // 是否 AI 起草 (vs PM 原创)  adopted_from_draft: bool  // 如果是 AI 起草, 是否被 PM adopt  linked_baskets: Basket[]  // 通过 ExpressionLink 关联的 baskets  created_at, updated_at }
```

4.3 Scenario 状态机

State	含义	进入条件	出口条件
draft	AI 起草或 PM 编辑中	AI generation / PM 创建	PM adopt → active
active	正在追踪的 scenario	PM adopt	Trigger → triggered; Falsification → invalidated

triggered	被验证（一个或多个 triggers 命中）	Trigger condition 命中	时间长或 context 变 → obsolete; evolve 为新 sub-scenario
invalidated	被证伪	Falsification signal 命中	只读（保留 lineage）
obsolete	时效过期或 theme 转向	PM 手动标注 or AI 建议	只读

4.4 AI 在 Scenarios 阶段的 function · 核心章节

这是 V5.5 framework 中 AI 最活跃、价值最大的阶段。AI 的每个动作都必须遵守 "expand option space, PM makes choices" 原则——AI 输出 candidates, PM 输出 decisions。

 **AI · Scenario Candidate Generation** 这是 AI 的核心动作。当 PM 创建一条新 thesis 后, AI 基于: (1) thesis 的 claim 和 causal chain; (2) parent theme 下的历史 scenarios 库; (3) 当前 market context (同类 thesis 的常见 paths) ——起草 5-8 条候选 scenarios。每条 scenario 包含初步的 narrative + trigger signals + candidate tickers。PM 审阅, adopt / dismiss / modify。

 **AI · Blind Spot Detector** AI 分析 PM 已 adopt 的 scenarios, 识别可能遗漏的维度——比如 PM 只考虑了 bull/bear 两条, AI 提示 "你没有考虑 supply shock scenario (原材料涨价导致整个行业成本转嫁)"。这是 expand option space 的核心体现。

 **AI · Cross-Thesis Scenario Linkage** AI 识别当前 scenario 是否可以共享 basket with 其他 theses 下的 scenarios。例: Theme A 下的 "custom silicon 崛起" scenario 与 Theme B 下的 "AI 软件股利好" scenario 可以 share 同一个 "Long AI software / Short GPU pure play" basket。这为 stage 4 (Baskets) 的 N:M 关系打基础。

 **AI · Scenario Stress Tester** PM adopt 某个 scenario 后, AI 主动挑战——"如果 X 发生, 这条 scenario 的 narrative 会不会失效?" PM 回答 "会", AI 建议把 X 加到 falsification condition; PM 回答 "不会", AI 建议强化 narrative 的 robustness 说明。

 **AI · Scenario Tree Navigator** UI 上 AI 辅助 PM 在 scenario tree 中导航: (1) 哪些 scenarios 的 trigger 即将命中 (AI 估算); (2) 哪些 scenarios 有 overlap 可以合并; (3) 哪些 active scenarios 长期没更新, 建议 review。

AI 不做什么 (必须 surface)

- **不替 PM 决定哪些 scenarios 进入 active state**——AI 起草, PM adopt
- **不替 PM 赋予 subjective probability**——这是 PM 的主观信心, AI 无法代替
- **不自动合并 scenarios**——即使 AI 检测到两条 scenarios 高度相似, 也只是 suggest, 由 PM 决定
- **不基于 AI 自己的市场观点生成 scenarios**——AI 的 scenario 生成基于 thesis 的 causal chain 展开, 不是 AI 自己的 prediction

4.5 产品设计要点

- **Scenario Tree Editor**: 核心界面。Thesis 为根节点, scenarios 为子节点。PM 原创节点 (深蓝) vs AI 起草节点 (浅蓝) 视觉区分。Triggered scenarios (绿色)、invalidated (红色) 状态一眼可见
- **AI Draft Tray**: AI 起草的候选 scenarios 在侧边栏, PM 可以拖入 tree (adopt) 或 dismiss
- **Probability Distribution View**: 显示同一 thesis 下所有 scenarios 的主观概率分布 (PM 输入), 可视化 PM 的信心结构

- **Blind Spot Notifications:** AI 定期 review PM 的 scenario set, 弹出 "你可能遗漏了 X 维度" 的 suggestion
- **Lineage View:** scenario evolve (sub-scenario 分叉) 的历史轨迹, 类似 git tree

Baskets 是 framework 中唯一可量化的对象。所有 PM 关心的 quant signals (PnL、Sharpe、attribution、factor exposure) 都挂在这里。Basket 与 Scenarios 是 N:M 关系——一个 basket 可表达多个 scenarios，一个 scenario 可衍生多个 baskets。

5.1 定义

Basket 是一组 **instruments + weights + entry rules**，作为 scenarios 的可执行表达。它承载 PM 的 portfolio 决策，也是 performance attribution 的归属单元。

Basket 与 Scenarios 的 N:M 关系

情况 A · 多个 scenarios 共享一个 basket	PM 有两个 scenarios: "AMD 挑战 NVDA" + "Custom silicon 崛起"。两者 basket 表达都是 "short NVDA + long 替代品"——可以共享一个 "NVDA underperform" basket
情况 B · 一个 scenario 有多个 basket 表达	Scenario "GLP-1 渗透加速" 可以有：(1) Long-only basket (做多受益方)；(2) Long-short basket (做多受益 + 做空受冲击)；(3) Options basket (用 calls 表达 upside)。PM 根据 risk budget 选择

5.2 Basket 对象

```
Basket {  id: UUID    name: string                                // "NVDA underperform - Long
alts"    description: string                                // basket 的 investment thesis 简述
instruments: [{    ticker: string,    weight: float,                                // 占 basket 的
权重    direction: enum,                                // long / short    entry_price: float,
target_price: float,    stop_loss: float,  }]    entry_rules: string                                //
"分 3 次建仓, 每次 1/3"    risk_budget: float                                // 这个 basket 占 PM 总 risk
budget 的比例    factor_exposure: {                                // 计算得出, 动态更新    sector_tilts:
dict,    beta: float,    momentum: float,    quality: float,  }    state: enum
// active / closed / scaling / hedging    performance: PerformanceRecord[] // 时间序列
created_at, updated_at, author_id } ExpressionLink {                                // N:M 关系表
id: UUID    scenario_id: UUID    basket_id: UUID    weight_contribution: float    //
这个 scenario 贡献给 basket 的权重 (0-1)    notes: string    created_at }
```

5.3 Basket 状态机

State	含义	进入条件	出口条件
draft	构建中, 未执行	PM 创建	PM activate → active

active	正在持仓，追踪 PnL	PM activate	关联 scenarios 全 triggered → closed; 全 invalidated → closed
scaling	加减仓中	PM 决定调整 weights	调整完成 → active
hedging	对冲中（某 scenario trigger 后 risk-off）	PM 临时 hedge	PM 移除 hedge → active
closed	已平仓，performance 定格	PM 决定 close	只读（保留 PnL lineage）

5.4 AI 在 Baskets 阶段的 function

 **AI · Candidate Instrument Recommender** PM 创建 basket 时，AI 基于关联 scenarios 的 narrative 推荐 candidate instruments。例：scenario 是 "custom silicon 崛起"，AI 推荐 long GOOGL (TPU 制造) + long AMZN (Trainium) + short NVDA。AI 给出推荐理由（与 narrative 的匹配度），PM 决定采纳哪些。

 **AI · Historical Analog Finder** AI 搜索历史上相似的 baskets（同一 PM、同公司其他 PM、或公开数据），显示它们的 PnL 表现、最大回撤、turnover。"你这个 basket 与 2022 Q3 的 XX basket 结构相似，那次 6 个月收益 +18%，最大回撤 -12%"。这给 PM 一个 empirical anchor。

 **AI · Factor Exposure Analyzer** 每次 PM 调整 basket weights 或 instruments，AI 实时计算 factor exposure（sector tilt、beta、momentum、quality），flag 超过 PM 预设阈值的暴露。"你当前 basket 的 semi 行业暴露 45%，超过你设定的 30% limit"。

 **AI · Cross-Basket Correlation Monitor** AI 监测 PM 所有 active baskets 之间的相关性。如果两个 basket 相关性 > 0.7（意味着实质上在重复下注），AI 建议合并或 rebalance。这与 Scenarios 阶段的 cross-thesis linkage 互补——Scenarios 层看 narrative overlap，Basket 层看 positional overlap。

 **AI · Attribution Engine** Basket 收益产生后，AI 自动归因：这个 PnL 来自哪些 instruments？哪些是 alpha（scenario-specific）哪些是 beta（市场）？关联的 scenarios 哪个 trigger 最显著贡献了 PnL？这是 framework 给 PM 的核心价值——让每一分 PnL 都有明确出处。

AI 不做什么

- 不替 PM 决定 weights（AI 推荐 instruments，PM 定 weights）
- 不替 PM 决定 entry/exit timing
- 不自动 trigger 交易（framework 不是 execution platform）
- 不基于 AI 的预测调整 weights（只有 trigger event 或 PM 决策能改变 basket）

5.5 产品设计要点

- **Basket Builder**: 核心编辑界面。左侧关联 scenarios（可多选，N:M），中间 instruments table（带 AI 推荐 tag），右侧实时 factor exposure + 历史 analog。PM 主导编辑，AI 提供 suggestions
- **Portfolio Aggregation View**: 所有 active baskets 的汇总视图，按 risk budget / scenario 关联 / sector 切片。跨 basket 相关性热力图
- **Attribution Dashboard**: 每个 basket 的 PnL breakdown——哪些来自 trigger 命中、哪些来自市场 beta、哪些来自 factor premium。这是 PM 的核心决策输入
- **Basket-Scenario Map**: 可视化 N:M 关系。显示每个 basket 链接哪些 scenarios（通过 ExpressionLink）、权重贡献比例

6.1 定义

Monitor & Evolve 是 loop 的 持续后台动作，同时在两个层面工作：

- **Monitor @ Scenario 层 · qualitative**——追踪每个 active scenario 的 trigger signals，检测 trigger 命中 / falsification 发生
- **Monitor @ Basket 层 · quantitative**——追踪每个 active basket 的 PnL、factor exposure、risk metrics
- **Evolve @ Thesis 层**——当某个 scenario 被大量 triggered 或 invalidated，触发 thesis vN+1 的起草；保留完整 lineage

6.2 Monitor 的双层结构

维度	Scenario 层 Monitor	Basket 层 Monitor
性质	Qualitative · event-based	Quantitative · continuous
追踪对象	Trigger signals (earnings、macro、industry events)	PnL、weights、factor exposure、turnover、Sharpe
触发频率	事件驱动 (sparse、不连续)	连续 (每日结算)
影响	Scenario state 变化 (active → triggered / invalidated)	Basket 的 PnL 累积、factor 监控、风险警报
回流到	Thesis 层 (evolve trigger)	Attribution report (performance review)

6.3 Evolve 机制

什么时候触发 Evolve

- **Scenario triggered with significance**：某个 scenario 的 trigger signals 命中，且 AI 或 PM 判断这改变了 thesis 的 causal chain
- **Multiple scenarios triggered simultaneously**：同一 thesis 下多个 scenarios 同时 trigger，意味着 thesis 的假设有待更新
- **Falsification 命中**：thesis 的 falsification condition 被触发，需要 evolve (modify claim) 或 invalidate
- **Long inactivity + new context**：thesis 长时间无新 trigger，但 parent theme 有重大 context 变化，建议 review

Evolve 产出

- Thesis vN+1 草案 (AI 起草, PM review)
- Sub-scenarios (新的分支 scenarios, 挂在 triggered scenario 下)
- Lineage record (完整保留 vN $\rightarrow$ vN+1 的变化轨迹)

6.4 AI 在 Monitor & Evolve 阶段的 function

 **AI · Trigger Signal Watcher** AI 7×24 扫描 market events、earnings、macro data、industry news。对每个 active scenario 的 trigger signals 做 pattern matching——命中时生成 TriggerEvent 记录，更新 scenario state，通知 PM。支持三种命中等级：pending（信号部分满足）/ triggered（完全满足）/ disproved（证据朝相反方向）

 **AI · Signal Quality Evaluator** 不是所有信号都等价。AI 评估 trigger 的可靠性——“这个 earnings miss 是由一次性因素导致还是结构性变化？”避免噪声触发 false positive。PM 可以设置 confidence threshold（e.g., only surface triggers with >80% confidence）

 **AI · Cross-Scenario Correlation Detector** 当多个 scenarios 在短时间内 trigger，AI 分析是否有共同 root cause——这通常是 thesis evolve 的信号。“你的 5 个 scenarios 中有 3 个在 Q4 earnings 后 triggered，都指向 data center 需求超预期——thesis 可能需要升级到 vN+1”

 **AI · Thesis Evolution Drafter** 当 evolve 条件满足，AI 起草 thesis vN+1：(1) 保留原 claim 框架；(2) 整合 triggered scenarios 的新证据到 causal chain；(3) 更新 falsification conditions。PM 审阅后决定 accept / reject / modify

 **AI · Performance Attribution Engine** 对每个 basket 的 PnL，AI 做 attribution：来自 triggered scenarios 的 alpha、来自市场 beta、来自 factor premium、来自 execution alpha。输出结构化的 attribution report，让 PM 知道“我赚的钱来自哪条 thesis 的验证”

 **AI · Retroactive Signal Discovery** 定期（每月）回头看：PM 的 triggered scenarios 之前有哪些早期信号没被识别？AI 学习这些 patterns，改进未来 trigger signal 的起草建议。这是 framework 的 “learning loop”

AI 不做什么

- 不自动修改 scenario state（trigger 命中后提示 PM 确认）
- 不自动发布 thesis vN+1（只起草，PM 决定）
- 不预测 trigger 何时会命中（只追踪，不 forecast）
- 不自动调整 basket weights（即使 scenario state 变化，也只是 suggest，PM 决策）

6.5 产品设计要点

- **Unified Monitor Dashboard**：双栏视图——左侧 scenario triggers timeline（事件驱动），右侧 basket performance（连续）。关键 triggers 和 PnL 异常 top surface
- **Evolve Drafts Inbox**：AI 起草的 thesis vN+1 和 sub-scenarios 聚合，PM review。每份 draft 显示触发原因（哪个 scenario triggered、什么 signal）
- **Attribution Reports**：定期（周 / 月 / 季）自动生成。按 theme / thesis / scenario / basket 切片显示 PnL attribution
- **Lineage Viewer**：git-like 时间线，显示 thesis 和 scenario 的演化历史

七、AI-Native 设计原则 · 整体纲领

前 6 章定义了每个阶段的 AI function。本章从整体角度 surface AI-native 的设计原则，避免局部优化导致的 drift。这些原则是跨阶段的，所有 AI 设计必须同时满足。

7.1 AI-Native 不是什么

AI-Native 不是	AI-Native 是
"在每个 feature 上加 AI 按钮"	AI 渗透到每个对象、状态转移、决策时机
"AI 替用户做决策"	AI expand option space, PM make choices
"AI 自动化所有重复工作"	AI 自动化信号检测 + lineage 管理；认知动作留给 PM
"LLM 作为前端包装"	AI 深度集成到 data model 和 state machine
"AI 根据自己的市场观点行动"	AI 基于 PM 的 thesis 展开；不产生独立 prediction

7.2 五条 AI-Native 原则

原则 A · Candidates, not Decisions

AI 在每个阶段的输出都是 **候选项 (candidates)**，不是决策 (decisions)。PM 是唯一的决策者。

具体表现：

- Theme Radar 起草 emerging themes → PM 决定 adopt
- Falsifiability Coach 给 feedback → PM 决定修改
- Scenario Candidate Generator 起草 5-8 条 → PM 决定 adopt 哪几条
- Instrument Recommender 推荐 tickers → PM 决定 weights
- Trigger Watcher 命中 signal → PM 确认 state 变化
- Evolution Drafter 起草 thesis vN+1 → PM 决定发布

原则 B · Proactive, not Reactive

AI 不是等 PM 主动查询才工作——它 **7×24 proactive 扫描**，发现可能相关的 signals、patterns、blind spots，主动推送 suggestions 到 PM 的 inbox。

具体表现：

- Theme Radar 定期扫描 market signals 推荐新 themes
- Blind Spot Detector 定期 review PM 的 scenario set
- Trigger Watcher 7×24 matching signals against active scenarios
- Cross-Basket Correlation Monitor 每日检查相关性漂移
- Retroactive Signal Discovery 每月回溯学习

原则 C · Structured, not Free-form

AI 的输出必须是 **结构化对象**，能直接进入 framework 的 data model——不是自由文本让 PM 自己解析。

具体表现：

- Scenario Candidate 输出的是 Scenario 对象（含 trigger signals、parent thesis 等字段），不是一段叙述
- Instrument Recommendation 输出的是结构化 ticker list（含 direction、rationale）
- Thesis Evolution Drafter 输出的是完整 Thesis vN+1 对象（含 version、lineage 链接）
- Attribution Report 是结构化的 breakdown，不是总结段落

原则 D · Explainable, not Black-box

每个 AI 输出都必须 **自解释**——PM 能看到 "AI 为什么这么建议"。没有 explanation 的输出等于没有输出。

具体表现：

- Scenario candidate 显示 "based on parent thesis causal chain X + historical analog Y"
- Instrument recommendation 显示 "matches scenario narrative keyword Z + recent performance pattern"
- Trigger event 显示 specific signal 和 confidence level
- Attribution 显示 decomposition math (alpha vs beta vs factor)

原则 E · Grounded, not Speculative

AI 不基于自己的市场预测行动。所有 AI 推荐必须 **grounded 在 framework 对象** (thesis、scenario narrative、historical data) 或可观测信号。

具体表现：

- Scenario generation 基于 thesis causal chain + 同类 thesis 历史 scenarios，不是 AI 自己的 market view
- Instrument recommendation 基于 scenario narrative + historical analog basket，不是 AI 预测股价
- Trigger event 基于真实 market data + PM 定义的 trigger condition，不是 AI 推测
- Evolution 基于 triggered scenarios 的证据累积，不是 AI "觉得" 该更新

7.3 AI 不跨越的红线

这些是绝对红线——任何 AI feature 设计如果违反其中一条，必须重新设计。红线是 framework 的护城河，保证 PM 始终是主导者。

- **红线 1 · 不替 PM 润色 thesis 语法**——thesis 是 PM 的认知表达，AI 语法改动会改变论断的微妙含义
- **红线 2 · 不替 PM 决定 basket weights**——weights 反映 PM 的 risk budget 分配，是核心决策
- **红线 3 · 不自动修改 scenario/thesis state**——即使 trigger 命中，也只是 surface notification，PM 确认后才变 state
- **红线 4 · 不产生独立市场预测**——AI 是 framework 内的执行者，不是独立的投资顾问
- **红线 5 · 不自动触发交易**——Framework 不是 execution platform
- **红线 6 · 不隐藏 reasoning**——所有 AI 输出必须可解释（原则 D），PM 始终理解 why

八、Forks · V5.5 的 Retail 实现

本章明确 Forks 与 V5.5 Framework 的关系。这是产品决策层的 clarification，防止 framework（面向专业 PM）和 Forks（面向 retail）两个层面的混淆。

8.1 两个层面的清晰分离

维度	V5.5 Framework	Forks (V5.5 retail 实现)
目标用户	专业 PM (buy-side)	fintwit、aspiring buyside、semi-pro、学生
用户核心 job	Performance attribution、PnL、风险管理	观点表达、社交分享、被验证的 credibility
Basket 是否独立	必须独立 first-class object	内嵌在 scenario card 的 "associated tickers"
Scenario ↔ Basket 关系	N:M (灵活表达)	简化为 1:1 (每 scenario 自带简单 basket)
Attribution 需求	完整 PnL / Sharpe / factor	无 (retail 用户不做 attribution)
社交层	无 (专业 PM 不做 social sharing)	Vindication card + scenario card 分享是核心

8.2 为什么 Forks 可以简化

Forks 的简化不是阉割，是产品决策——retail 用户的 complexity tolerance 不同，需要不同的产品形态承载同样的 framework 核心思想。

- **Retail 用户不做 performance attribution**，所以不需要独立 Basket 对象支持 PnL 归因
- **Retail 用户需要低 friction 分享**，所以 basket 信息内嵌在 scenario card（一张图搞定）而不是独立页面
- **Retail 用户的 job 是表达观点 + 社交可信度**，所以 vindication card（trigger 命中后的分享）是核心，而不是 quant dashboard

8.3 Forks 对 v5.5 的 mapping

V5.5 阶段	Framework 实现	Forks 实现
---------	--------------	----------

Theme	Theme dashboard + Theme Radar	Hashtag / topic (轻量)
Thesis	完整 Thesis editor + Falsifiability Coach	极简一句话 (tree root)
Scenarios	Scenario tree + full AI functions	Scenario tree + AI expansion (MVP 核心)
Baskets	独立 Basket 对象 + N:M + factor analysis	Scenario 内嵌 associated tickers (1:1 简化)
Monitor & Evolve	双层 Monitor + Evolve Drafter + Attribution	Trigger Watcher + Vindication card 自动生成

8.4 演化路径

V5.5 给 Forks 一个明确的演化路径：当用户成熟到需要 institutional 级功能时，可以推出 Forks Pro tier——暴露完整 Basket 对象、N:M 关系、factor analysis。此时 Forks Pro 就成为 framework 的完整实现，服务专业用户。

这个路径的价值：不需要重新设计 framework，只是把已有定义渐进 expose 给成长中的用户。

九、Schema & API 规范

9.1 核心表结构 (PostgreSQL 参考)

```
-- Theme CREATE TABLE themes ( id UUID PRIMARY KEY, name VARCHAR(200) NOT NULL,
description TEXT, state theme_state_enum DEFAULT 'emerging', trigger_context
JSONB, author_id UUID, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at
TIMESTAMPTZ DEFAULT NOW() ); -- Thesis (with versioning) CREATE TABLE theses ( id
UUID PRIMARY KEY, parent_theme_id UUID REFERENCES themes(id), claim TEXT NOT
NULL, causal_chain TEXT, falsification_conditions JSONB, version INTEGER
DEFAULT 1, predecessor_id UUID REFERENCES theses(id), state thesis_state_enum
DEFAULT 'active', pm_confidence FLOAT, author_id UUID, created_at TIMESTAMPTZ,
updated_at TIMESTAMPTZ ); -- Scenario CREATE TABLE scenarios ( id UUID PRIMARY
KEY, parent_thesis_id UUID REFERENCES theses(id), narrative TEXT NOT NULL,
trigger_signals JSONB, pm_probability FLOAT, state scenario_state_enum DEFAULT
'draft', ai_generated BOOLEAN DEFAULT FALSE, adopted_from_draft BOOLEAN DEFAULT
FALSE, author_id UUID, created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ ); --
Basket (first-class object) CREATE TABLE baskets ( id UUID PRIMARY KEY, name
VARCHAR(200) NOT NULL, description TEXT, instruments JSONB, -- array of
{ticker, weight, direction, entry, target, stop} entry_rules TEXT, risk_budget
FLOAT, factor_exposure JSONB, -- computed, dynamically updated state
basket_state_enum DEFAULT 'draft', author_id UUID, created_at TIMESTAMPTZ,
updated_at TIMESTAMPTZ ); -- ExpressionLink (N:M Scenario ↔ Basket) CREATE TABLE
expression_links ( id UUID PRIMARY KEY, scenario_id UUID REFERENCES
scenarios(id), basket_id UUID REFERENCES baskets(id), weight_contribution FLOAT,
notes TEXT, created_at TIMESTAMPTZ DEFAULT NOW(), UNIQUE(scenario_id, basket_id)
); -- Trigger Events CREATE TABLE trigger_events ( id UUID PRIMARY KEY,
scenario_id UUID REFERENCES scenarios(id), signal_matched JSONB, severity FLOAT,
confidence FLOAT, pm_acknowledged BOOLEAN DEFAULT FALSE, created_at TIMESTAMPTZ
DEFAULT NOW() ); -- Performance Records (quant attribution) CREATE TABLE
performance_records ( id UUID PRIMARY KEY, basket_id UUID REFERENCES
baskets(id), timestamp TIMESTAMPTZ, pnl FLOAT, attribution JSONB, -- {alpha,
beta, factor_premium, execution} factor_snapshot JSONB, created_at TIMESTAMPTZ
DEFAULT NOW() );
```

9.2 关键 API endpoints

# Theme GET /themes	# List with filters POST /themes
# Create PATCH /themes/{id}	# Update state, description GET
/themes/{id}/lineage	# Full lineage tree # Thesis (with versioning) GET
/theses	# List POST /theses # Create v1
POST /theses/{id}/evolve	# Create vN+1 (AI-drafted or manual) GET
/theses/{id}/versions	# All versions PATCH /theses/{id}/state #

```

active → evolving → etc. # Scenarios GET /scenarios # List with
thesis_id filter POST /scenarios # Manual create POST
/scenarios/ai-draft # AI generate candidates PATCH /scenarios/{id}/adopt
# Adopt AI draft PATCH /scenarios/{id}/probability # Update PM subjective
probability # Baskets (first-class) GET /baskets # List all
POST /baskets # Create PATCH /baskets/{id} #
Update instruments, weights POST /baskets/{id}/link-scenario # Add ExpressionLink
DELETE /baskets/{id}/link-scenario/{scenario_id} GET /baskets/{id}/performance #
Time series PnL GET /baskets/{id}/attribution # Decomposition # ExpressionLinks
(N:M) GET /expression-links?scenario_id=X # All baskets for scenario GET
/expression-links?basket_id=Y # All scenarios for basket # Monitor (trigger
events) GET /triggers # Recent events POST
/triggers/{id}/acknowledge # PM confirms state change # Evolve GET
/evolve/drafts # AI-drafted thesis vN+1 POST
/evolve/drafts/{id}/accept # Accept, publish new version POST
/evolve/drafts/{id}/reject # Reject, keep current # AI Interfaces POST
/ai/scenarios/generate # Generate candidates for thesis POST
/ai/baskets/recommend # Recommend instruments for scenario POST
/ai/theses/falsifiability # Check thesis structure GET /ai/blind-spots
# Detected blind spots in PM's scenarios GET /ai/correlations # Cross-
basket correlation warnings

```

9.3 AI Service 接口规范

AI 功能通过独立 service 暴露，与 framework 主服务解耦。每个 AI endpoint 必须满足：

- 输出是结构化对象（原则 C）
- 包含 reasoning/explanation 字段（原则 D）
- 明确标记为 candidate / suggestion / draft（原则 A）
- grounded 到具体 framework 对象（原则 E）

```

# AI Scenario Generation Response Format {  "candidates": [    {      "scenario":
{      "narrative": "...",      "trigger_signals": [...],
"suggested_tickers": [...],      },      "reasoning":
{      "based_on_thesis_claim": "...",      "historical_analogs": [...],
"confidence": 0.75      },      "metadata": {      "generated_at": "...",
"model_version": "...",      "prompt_version": "..."      }      },      ...  ] }

```